Tema 6

Desarrollo de clases

Programación

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Concepto de clase 5
3. Estructura y miembros de una clase 6
4. Creación de atributos. Declaración e inicializción 7
5. Creación de métodos. Declaración, argumentos y valores de retorno 8
6. Creación de constructores 9
7. Ámbito de atributos y variables 11
8. Sobrecarga de métodos 14
9. Visibilidad. Modificadores de clase, de atributos y de métodos 17
10. Paso de parámetros. Paso por valor y paso por referencia 19
11. Utilización de clases y objetos 21
12. Utilización de clases heredadas 22
13. Librerías y paquetes de clases. Utilización y creación 25
13.1 Módulo 25
13.2 Librería 25
12.3 Paquetes 27
14. Documentación sobre librerías y paquetes de clases 29
14.1 Tipos de librerías en programación 29
14.2 Librerías estáticas 29
14.3 Librerías dinámicas 29
14.4 Bibliotecas en programación más usadas 29
15. Resumen 31
Glosario 32
Bibliografía 34

Objetivos de la unidad

En esta unidad se pretende que el alumno tenga una primera toma de contacto con la programación orientada a objetos y consiga los siguientes objetivos:

- Comprender los conceptos fundamentales de la Programación Orientada a Objetos (POO): Este objetivo busca establecer una base sólida de conocimientos sobre los principios y conceptos centrales de la POO, como clases, objetos, encapsulamiento, herencia, polimorfismo, entre otros.

- Diseñar y crear clases en un lenguaje de programación orientado a objetos: El objetivo es proporcionarnos las habilidades necesarias para diseñar y desarrollar clases en un lenguaje de programación orientado a objetos específico. Se enfocará en la sintaxis y estructura de las clases, así como en la definición de atributos y métodos.

- Aplicar principios de encapsulamiento y modularidad en el diseño de clases: Este objetivo se centra en cómo utilizar el encapsulamiento para ocultar la implementación interna de una clase y exponer solo la interfaz necesaria. También se abordará la importancia de la modularidad en el diseño de clases, dividiendo la funcionalidad en módulos coherentes y reutilizables.

- Utilizar herencia y polimorfismo para extender y reutilizar clases: Aquí se busca cómo utilizar la herencia para crear nuevas clases basadas en clases existentes, aprovechando la funcionalidad y características heredadas. También se abordará el polimorfismo, permitiendo a los objetos responder de manera diferente a una misma llamada de método.

- Aplicar buenas prácticas de codificación en la programación orientada a objetos: Aquí se enfocará en promover el uso de buenas prácticas de codificación, como la nomenclatura adecuada de variables y métodos, la documentación clara y concisa, la organización del código, entre otros aspectos relevantes para mantener un código limpio y legible.

- Analizar y comprender el impacto de la programación orientada a objetos en el desarrollo de software: Este objetivo busca brindar una perspectiva más amplia sobre los beneficios y las implicaciones de la programación orientada a objetos en el desarrollo de software, incluyendo temas como la reutilización de código, la escalabilidad, el mantenimiento y la colaboración en equipo.

1. Introducción

Uno de los paradigmas más populares y poderosos de la programación es la Programación Orientada a Objetos (POO). La POO es un enfoque de diseño y desarrollo de software que se basa en la creación de clases y objetos, permitiendo una mayor modularidad, reutilización de código y estructuras más flexibles.

De este modo, se pueden diseñar y desarrollar clases en un lenguaje de programación orientado a objetos. Es preciso, para ello, comprender los pilares de la POO, la encapsulación, la herencia y la sobrecarga de métodos y, además, cómo aplicarlos en la creación de nuestras propias clases.

Existen diversas técnicas y mejores prácticas para el diseño y desarrollo de clases. La más importante es la modularidad, el reuso de código y la estructuración clara de las responsabilidades.

Otro concepto importante es la importancia del diseño orientado a objetos, identificando los principios clave que nos ayudarán a crear un código limpio, legible y fácilmente mantenible.

Hay que tener en cuenta las implicaciones y ventajas de la POO en el desarrollo de software, desde la reutilización de código hasta la escalabilidad y el trabajo en equipo. Adquirir estas habilidades es fundamental para convertirse en un desarrollador de software competente y poderoso.

2. Concepto de clase

Una **clase** es una **representación o plantilla** para muchos objetos del mismo tipo. Sirve como plantilla para todos los objetos de un tipo particular, definiendo variables y métodos comunes para cada uno. Para el objeto coche definido a continuación, una clase podría incluir los siguientes componentes:

AUTO
Marca String
Color String
Matricula String
Modelo String
cuenta_kilometros Integer
Velocidad Integer

Una vez declarada la clase, es posible crear objetos a partir de esta como si fuese una plantilla. La creación de objetos a partir de una clase o plantilla se denomina instanciación. Dado que cada uno de los objetos o instancias tiene una copia única de los atributos y métodos, la asignación de valor es completamente independiente de las demás operaciones.

3. Estructura y miembros de una clase

A partir de la clase definida en el apartado anterior vamos a definir los conceptos de estructura y miembros de una clase:

Definición

atributos

Las características unívocas que permiten distinguir los objetos de una clase u otra.

métodos:

Las operaciones o funciones que especifican cómo se comportan los objetos pertenecientes a una clase se denominan métodos.

Para una clase de tipo persona definiremos los atributos sombreados en azul y los métodos sombreados en amarillo:

```
class Person:
    name = ''
    school = ''

    def print_name(self):
        print self.name

    def print_school(self):
        print self.school

jorge = Person() ; # instanciamos un objeto de tipo Person
jorge.name = 'Jorge'                        #atributo
jorge.school = 'Universidade de Vigo'       #atributo
jorge.print_name()                          #método
jorge.print_school()                        #método
```

4. Creación de atributos. Declaración e inicializción

Siguiendo con el ejemplo del apartado 1 de la unidad tomamos la clase Coche:

A continuación, vamos a poner un ejemplo de clase llamada Coche en Python

class Coche:

```
def __init__(self, marca, color, matrícula, modelo,
cuenta_kilometros, velocidad):
    # Propiedades
    self.numero_llantas = 4 # es una constante, siempre
habrá 4 llantas.
    self.marca = marca
    self.color = color
    self.matrícula = matrícula
    self.modelo = modelo
    self.cuenta_kilometros = 0
    self.velocidad = 0
```

5. Creación de métodos. Declaración, argumentos y valores de retorno

Siguiendo con el mismo ejemplo tomamos la clase Coche, a continuación, definiremos los métodos de la clase coche que definirán el comportamiento que tendrán los objetos instanciados de esa clase.

Después del nombre del método recibiremos los argumentos del método, en caso de no tener ningún argumento se indica con la palabra reservada **self**.

```
def acelerar(self):
      print(f"Soy {self.marca}, estoy acelerando!!")
  def frenar(self):
      print(f"Soy {self.marca}, estoy frenando!!")
  def girar(self, direccion):
      print(f"Soy {self.marca}, estoy girando a la
{direccion}")
  def pitar(self):
      print('Bip Bip Bip')
  def consultar_velocimetro(self):
      return self.velocidad
  def consultar_cuenta_kilometros(self):
      return self.cuenta_kilometros
```

Con la instrucción **return** devolveremos los valores de retorno del método

6. Creación de constructores

Python usa un método especial llamado **constructor** para crear un objeto usando las definiciones de su clase. Python crea la inicialización (asignación de valores a las variables) y otras tareas de inicio necesarias utilizando este constructor. Además, los constructores pueden confirmar que un objeto tiene recursos suficientes para realizar cualquier tarea inicial.

El constructor siempre tiene el mismo nombre, como __init__(). Cuando es necesario crear objetos, el constructor puede tomar argumentos. Python crea automáticamente un constructor inútil cuando se puede crear una clase sin necesidad de una. Incluso si una clase solo usa el constructor predeterminado, aún debe tener su propio constructor.

Para definir un constructor utilizamos la instrucción def _ _ init_ _ (self) y lo construimos de la siguiente forma:

```
def _ _init_ _ (self, numero):
if isinstance (numero, int):
```

Demostraremos cómo crear un constructor en los siguientes pasos:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print(f"¡Hola! Mi nombre es {self.nombre} y tengo
{self.edad} años.")
```

En este ejemplo, hemos definido la clase `Persona` con dos atributos: `nombre` y `edad`. El constructor de la clase se define mediante el método especial `__init__`. El primer parámetro del constructor es `self`, que hace referencia al objeto mismo que está siendo creado. Luego, definimos los parámetros `nombre` y `edad` que se asignarán a los atributos correspondientes del objeto.
Dentro del cuerpo del constructor, utilizamos la sintaxis `self.atributo = valor` para asignar los valores de los parámetros a los atributos de la instancia.

Una vez que se ha creado el constructor, podemos crear objetos de la clase `Persona` utilizando la siguiente sintaxis:

```
persona1 = Persona("Juan", 30)
```

En este caso, se crea un objeto `persona1` con nombre "Juan" y edad 30. Luego, podemos llamar al método `saludar()` en el objeto creado:

```
persona1.saludar()
```

Esto imprimirá el mensaje "¡ Hola! Mi nombre es Juan y tengo 30 años." en la consola.

7. Ámbito de atributos y variables

- **Atributos**:

La capacidad de definir variables y funciones dentro de las clases, aquí conocidas como atributos y métodos, respectivamente, es quizás el mejor ejemplo del potencial de la programación orientada a objetos.

En términos prácticos, la distinción principal entre atributos y variables es que los atributos solo existen dentro del objeto, mientas que las variables existen dentro y fuera del objeto.

- **Atributos dinámicos**:

Debido a la extrema flexibilidad de Python, los atributos se pueden usar de varias maneras

Los atributos dinámicos son aquellos que se pueden agregar, modificar o eliminar en tiempo de ejecución a un objeto, incluso si no están definidos en la clase.

Se puede agregar atributos dinámicos a un objeto simplemente asignando un valor a un nuevo nombre de atributo en el objeto. Esto crea un nuevo atributo que está específicamente asociado con ese objeto en particular

```
class Galleta:
    pass

galleta = Galleta()
galleta.sabor = "salado"
galleta.color = "marrón"

print(f"El sabor de esta galleta es {galleta.sabor} "
      f"y el color {galleta.color}")
```

Sabías que

Los atributos dinámicos son útiles cuando necesitamos agregar información adicional a un objeto en tiempo de ejecución, sin modificar la definición de la clase. Sin embargo, es importante tener en cuenta que los atributos dinámicos son específicos de cada instancia y no se comparten entre diferentes instancias de la clase.

En Python, hay dos tipos de variables en el contexto de las clases: **variables de clase** y **variables de instancia**.

- **Variables de clase**:

Definición

Son variables que se definen dentro de la clase y están asociadas a la clase en sí, en lugar de estar vinculadas a una instancia específica de la clase.

Se declaran fuera de cualquier método de la clase, generalmente al comienzo de la definición de la clase, y se comparten entre todas las instancias de esa clase. Las variables de clase se acceden utilizando el nombre de la clase o una instancia de la clase.

Ejemplo:

```
class MiClase:
    variable_de_clase = 10

    def __init__(self, instancia_variable):
        self.instancia_variable = instancia_variable
```

En este ejemplo, `variable_de_clase` es una variable de clase y `instancia_variable` es una variable de instancia.

Para acceder a la variable de clase:

```
print(MiClase.variable_de_clase)  # Salida: 10
```

- **Variables de instancia:**

Definición

Son variables que están vinculadas a una instancia específica de una clase.

Cada instancia de la clase puede tener sus propias copias de las variables de instancia. Se definen dentro del método especial `__init__()` de la clase y se acceden utilizando la sintaxis `self.nombre_variable` dentro de los métodos de la clase.

Ejemplo:

```
class MiClase:
 def __init__(self, instancia_variable):
     self.instancia_variable = instancia_variable
```

En este ejemplo, `instancia_variable` es una variable de instancia.

Para acceder a la variable de instancia:

```
obj = MiClase(20)
print(obj.instancia_variable)  # Salida: 20
```

Cada instancia de la clase puede tener un valor diferente para la variable de instancia.

En resumen, las variables de clase se comparten entre todas las instancias de la clase y se definen fuera de los métodos, mientras que las variables de instancia son específicas de cada instancia y se definen dentro del método `__init__ () ` de la clase.

8. Sobrecarga de métodos

En Python, la sobrecarga de métodos no se maneja de la misma manera que en otros lenguajes de programación como Java o C++, ya que Python no admite la definición de varios métodos con el mismo nombre en una misma clase. En su lugar, se pueden utilizar argumentos predeterminados en la definición del método para simular la sobrecarga.

Por ejemplo, supongamos que queremos definir una clase Calculadora que tenga un método sumar que pueda tomar uno o dos argumentos. Podríamos definir la clase de la siguiente manera: Veamos un ejemplo:

```
class Calculadora:
    def sumar(self, a, b=None):
        if b:
            return a + b
        else:
            return a
```

En este ejemplo, el método sumar tiene dos parámetros: a y b. Sin embargo, el parámetro b tiene un valor predeterminado de None, lo que significa que, si se llama al método con solo un argumento, b se establecerá automáticamente en None.

Dentro del método, se verifica si b tiene un valor, lo que indica que se pasaron dos argumentos, y en ese caso se realiza la suma de a y b. Si solo se pasó un argumento, el método devuelve simplemente ese valor.

De esta manera, podemos llamar al método sumar de la siguiente manera:

```
calc = Calculadora()
# Llamada al método con un solo argumento
print(calc.sumar(5)) # Salida: 5
# Llamada al método con dos argumentos
print(calc.sumar(5, 10)) # Salida: 15
```

Importante

La sobrecarga de métodos en Python se refiere a la capacidad de definir múltiples métodos con el mismo nombre, pero con diferentes parámetros en una clase. Esto permite que una clase tenga varios métodos con el mismo nombre, pero con comportamientos diferentes según los argumentos que reciban.

Python no admite la sobrecarga de métodos de la misma manera que otros lenguajes de programación como Java o C++. Sin embargo, se puede simular la sobrecarga de métodos utilizando argumentos opcionales o utilizando argumentos de variable arbitraria (*args, **kwargs).

Aquí vemos un ejemplo de cómo simular la sobrecarga de métodos en Python utilizando argumentos opcionales:

```
class MiClase:
    def mi_metodo(self, arg1, arg2=None):
        if arg2 is None:
            # Comportamiento cuando solo se pasa arg1
            print("Se pasó solo arg1:", arg1)
        else:
            # Comportamiento cuando se pasa arg1 y arg2
            print("Se pasaron arg1 y arg2:", arg1, arg2)

obj = MiClase()

obj.mi_metodo(10)  # Salida: Se pasó solo arg1: 10
obj.mi_metodo(10, 20)  # Salida: Se pasaron arg1 y arg2: 10
20
```

En este ejemplo, el método `mi_metodo` se define con dos parámetros, `arg1` y `arg2`, siendo `arg2` opcional con un valor predeterminado de `None`. Dependiendo de si se proporciona o no el argumento `arg2`, el método se comporta de manera diferente.

También puedes simular la sobrecarga de métodos utilizando argumentos de variable arbitraria (`*args` y `**kwargs`), lo que permite pasar un número variable de argumentos:

```
class MiClase:
    def mi_metodo(self, *args):
        if len(args) == 1:
            # Comportamiento cuando se pasa un solo
argumento
            print("Se pasó un solo argumento:", args[0])
```

```
        else:
            # Comportamiento cuando se pasan varios
argumentos
            print("Se pasaron varios argumentos:", args)

obj = MiClase()

obj.mi_metodo(10)  # Salida: Se pasó un solo argumento: 10
obj.mi_metodo(10, 20, 30)  # Salida: Se pasaron varios
argumentos: (10, 20, 30)
```

En este caso, el método `mi_metodo` acepta cualquier número de argumentos utilizando `*args`. Luego, dentro del método, se puede verificar la longitud de `args` para determinar cómo se deben manejar los argumentos recibidos.

Hay que recordar que, aunque Python no tiene una sobrecarga de métodos estricta como en otros lenguajes, se pueden lograr comportamientos similares utilizando argumentos opcionales o argumentos de variable arbitraria.

9. Visibilidad. Modificadores de clase, de atributos y de métodos

Definición

En el contexto de programación orientada a objetos, la visibilidad en Python se refiere al alcance o accesibilidad de los atributos y métodos de una clase desde diferentes partes del programa.

Python proporciona tres niveles de visibilidad: **público**, **protegido** y **privado**. Estos niveles se logran mediante el uso de convenciones de nomenclatura y ciertos mecanismos de acceso.

- **Visibilidad pública:** Los atributos y métodos públicos son aquellos que se pueden acceder desde cualquier parte del programa, tanto desde dentro de la clase como desde instancias de la clase. Para lograr la visibilidad pública, se utiliza una convención de nomenclatura simple, en la que los atributos y métodos se definen sin un prefijo especial. Por ejemplo:

```
class MiClase:
    def __init__(self):
        self.atributo_publico = 10

    def metodo_publico(self):
        return "Método público"
```

En este caso, el atributo `atributo_publico` y el método `metodo_publico` son visibles y accesibles desde cualquier parte del programa.

- **Visibilidad protegida:** Los atributos y métodos protegidos son aquellos que están destinados a ser utilizados solo dentro de la clase o sus subclases. Para indicar que un atributo o método es protegido, se utiliza una convención de nomenclatura con un solo guion bajo al comienzo del nombre. Aunque Python no impide el acceso a estos elementos desde fuera de la clase, se considera una buena práctica no hacerlo. Por ejemplo:

```
class MiClase:
    def __init__(self):
        self._atributo_protegido = 20
```

```
    def _metodo_protegido(self):
        return "Método protegido"
```

En este caso, el atributo `_atributo_protegido` y el método `_metodo_protegido` son técnicamente accesibles desde fuera de la clase, pero se espera que se utilicen solo dentro de la clase o en subclases.

- **Visibilidad privada:** Los atributos y métodos privados son aquellos que se consideran estrictamente internos de la clase y no deben ser accedidos desde fuera de ella, ni siquiera por subclases. Para indicar que un atributo o método es privado, se utiliza una convención de nomenclatura con dos guiones bajos al comienzo del nombre. Aunque Python no impone restricciones de acceso, se considera una convención que los miembros privados no se accedan directamente desde fuera de la clase. Por ejemplo:

```
class MiClase:
    def __init__(self):
        self.__atributo_privado = 30

    def __metodo_privado(self):
        return "Método privado"
```

En este caso, el atributo `__atributo_privado` y el método `__metodo_privado` tienen una forma de "name mangling" (transformación de nombres) que hace que sean más difíciles de acceder desde fuera de la clase.

Importante

Es importante destacar que la visibilidad en Python se basa en convenciones y buenas prácticas, ya que el lenguaje no restringe el acceso directamente. Por lo tanto, es responsabilidad nuestra respetar estas convenciones para mantener una estructura de código clara y predecible.

10. Paso de parámetros. Paso por valor y paso por referencia

En Python, el paso de parámetros se realiza por asignación de referencia. Esto significa que cuando se pasa un parámetro a una función, se pasa la referencia al objeto en memoria y no una copia del objeto en sí. Aunque técnicamente no existe el paso de parámetros por valor en Python, es posible lograr un comportamiento similar dependiendo del tipo de dato con el que se esté trabajando.

- **Paso de parámetros inmutable (por valor):** Los tipos de datos inmutables, como números, cadenas de texto y tuplas, se comportan de manera similar al paso por valor. Al pasar un parámetro inmutable a una función, se crea una copia local de la referencia al objeto. Esto significa que cualquier modificación realizada dentro de la función no afectará al objeto original fuera de la función. Por ejemplo:

```
def modificar_numero(num):
    num += 1
    print("Valor dentro de la función:", num)

numero = 5
modificar_numero(numero)
print("Valor fuera de la función:", numero)

Output:
Valor dentro de la función: 6
Valor fuera de la función: 5
```

En este caso, el valor de `numero` no se modifica fuera de la función, ya que se crea una copia local del valor dentro de la función.

- **Paso de parámetros mutable (por referencia):** Los tipos de datos mutables, como listas y diccionarios, se pasan por referencia. Esto significa que cualquier modificación realizada dentro de la función afectará al objeto original fuera de la función. Por ejemplo:

```
def modificar_lista(lista):
    lista.append(4)
    print("Lista dentro de la función:", lista)

mi_lista = [1, 2, 3]
modificar_lista(mi_lista)
print("Lista fuera de la función:", mi_lista)
```

```
Output:
Lista dentro de la función: [1, 2, 3, 4]
Lista fuera de la función: [1, 2, 3, 4]
```

En este caso, la lista original `mi_lista` se modifica tanto dentro como fuera de la función, ya que se está trabajando con una referencia al mismo objeto.

Importante

Es importante tener en cuenta estas diferencias al pasar parámetros a funciones en Python, especialmente cuando se trabaja con objetos mutables. Si deseamos evitar modificaciones inesperadas en objetos mutables, podemos realizar copias explícitas dentro de la función para trabajar con una nueva referencia al objeto.

11. Utilización de clases y objetos

En la programación orientada a objetos una clase es una plantilla o molde que define la estructura y el comportamiento de los objetos que se van a crear. Representa un concepto o entidad con características y acciones específicas.

Un **objeto** es una estructura de datos personalizada que contiene datos y código:

Elementos	¿Qué son?	¿Cómo se llaman?	¿Cómo se identifican?
Datos	Variables	Atributos	Mediante sustantivos
Código	Funciones	Métodos	Mediante verbos

Un **objeto** representa una instancia única de alguna **clase** (a través de los valores de sus atributos) e interactúa con otros **objetos** (o consigo mismo) a través de sus **métodos**.

12. Utilización de clases heredadas

Al usar la herencia, podemos crear una clase secundaria que comparta métodos y atributos con una clase principal. Además, una clase secundaria tiene la capacidad de redefinir cualquier método o atributo existente, así como anularlos.

Definición

La herencia de clases es un principio esencial en la programación orientada a objetos (POO) que posibilita la generación de clases adicionales tomando como referencia clases previas ya existentes.

En Python, la herencia se implementa utilizando la sintaxis de la siguiente manera:

```
class ClaseBase:
    # Código de la clase base

class ClaseDerivada(ClaseBase):
    # Código de la clase derivada
```

En este ejemplo, `ClaseDerivada` es una clase que hereda de `ClaseBase`. La clase derivada adquiere todos los atributos y métodos de la clase base y también puede agregar nuevos atributos y métodos o modificar los existentes.

La herencia en Python puede ser tanto simple como múltiple. A continuación, se proporcionan ejemplos de ambos tipos de herencia.

- **Herencia Simple:**

En la herencia simple, una clase solo puede heredar de una clase base. Ejemplo:

```
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre
```

```
    def hacer_sonido(self):
        pass

class Perro(Animal):
    def hacer_sonido(self):
        return "Woof!"

class Gato(Animal):
    def hacer_sonido(self):
        return "Meow!"

mi_perro = Perro("Max")
print(mi_perro.nombre)  # Salida: Max
print(mi_perro.hacer_sonido())  # Salida: Woof!

mi_gato = Gato("Lucy")
print(mi_gato.nombre)  # Salida: Lucy
print(mi_gato.hacer_sonido())  # Salida: Meow!
```

En este ejemplo, la clase `Animal` es la clase base que tiene un atributo `nombre` y un método `hacer_sonido()`. La clase `Perro` y la clase `Gato` son clases derivadas que heredan de la clase `Animal`. Ambas clases derivadas implementan su propio método `hacer_sonido()` y pueden acceder al atributo `nombre` definido en la clase base.

- **Herencia Múltiple:**

En la herencia múltiple, una clase puede heredar de múltiples clases base. Ejemplo:

```
class Volador:
    def volar(self):
        return "Estoy volando"
```

```
class Nadador:
    def nadar(self):
        return "Estoy nadando"

class Pato(Volador, Nadador):
    pass

donald = Pato()
print(donald.volar())  # Salida: Estoy volando
print(donald.nadar())  # Salida: Estoy nadando
```

En este ejemplo, tenemos las clases `Volador` y `Nadador`, ambas con métodos específicos. La clase `Pato` hereda de ambas clases base utilizando la sintaxis de herencia múltiple. Como resultado, un objeto de la clase `Pato` tiene acceso tanto al método `volar()` como al método `nadar()`.

13. Librerías y paquetes de clases. Utilización y creación

13.1 Módulo

Definición

Podemos definir un **módulo** como una porción de un programa de ordenador. En el lenguaje de programación Python, un módulo se refiere a un archivo que incluye diversas definiciones y declaraciones de elementos propios de Python, tales como funciones, clases y variables.

Los módulos permiten organizar el código en archivos separados para facilitar la reutilización y el mantenimiento. Por lo general, cada archivo de Python es un módulo, y su nombre coincide con el nombre del archivo (con la extensión .py).

En Python, una librería (también conocida como biblioteca o módulo) es un conjunto de funciones, clases y métodos predefinidos que proporcionan funcionalidades adicionales para facilitar el desarrollo de aplicaciones. Las librerías en Python están diseñadas para abordar diferentes áreas, como manipulación de datos, matemáticas, visualización, redes, desarrollo web, entre otras.

13.2 Librería

Así podemos definir una **librería** como un conjunto de módulos cuya unión tiene una finalidad específica y que también puede ser invocada, como con un módulo.

Veamos un ejemplo de cómo crear y utilizar una **librería** en Python:

Supongamos que quieres crear una librería llamada "operaciones_matematicas" que contiene funciones para realizar operaciones matemáticas básicas. Primero, crea un archivo llamado `operaciones_matematicas.py` y escribe el siguiente código:

```
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    return a / b
```

En este código, hemos definido cuatro funciones simples para realizar operaciones matemáticas básicas.

Ahora, para utilizar esta librería en otro archivo Python, puedes hacer lo siguiente:

```
from operaciones_matematicas import sumar, restar, multiplicar, dividir

resultado_suma = sumar(5, 3)

resultado_resta = restar(7, 2)

resultado_multiplicacion = multiplicar(4, 6)

resultado_division = dividir(10, 2)


print("Suma:", resultado_suma)

print("Resta:", resultado_resta)

print("Multiplicación:", resultado_multiplicacion)

print("División:", resultado_division)
```

En este ejemplo hemos importado las funciones `sumar`, `restar`, `multiplicar` y `dividir` desde la librería "operaciones_matematicas". Luego,

utilizamos estas funciones para realizar diferentes operaciones matemáticas y mostramos los resultados en la consola.

Recordemos que tanto el archivo `operaciones_matematicas.py` como el archivo desde el cual lo importas deben estar en el mismo directorio o en una ubicación accesible desde el archivo que los importa.

Este es solo un ejemplo sencillo de cómo crear y utilizar una librería en Python. En la práctica, las librerías pueden contener muchas más funciones y funcionalidades útiles.

12.3 Paquetes

Definición

Un paquete es una forma de organizar y agrupar módulos relacionados. Un paquete es un directorio que contiene uno o varios archivos de módulo de Python, junto con un archivo especial llamado __init__.py.

El archivo __init__.py indica que el directorio es un paquete de Python y puede contener código de inicialización para el paquete. La estructura básica de un paquete en Python se ve así:

```
paquete/
  __init__.py
  modulo1.py
  modulo2.py
```

En este ejemplo, "paquete" es el nombre del paquete y "modulo1.py" y "modulo2.py" son los módulos que pertenecen a ese paquete. Debe contener siempre un archivo __init__.py (por el momento vacío) para que Python entienda que se trata de un paquete y no de una simple carpeta. De esta forma, podemos acceder a alguno de los módulos del paquete de la siguiente manera.

```
importpaquete.modulo1
print(paquete. modulo1.sumar(7, 5))
```

O bien de la siguiente manera.

```
from paquete import modulo1
print(modulo1.sumar(7, 5))
```

También, esta otra forma:

```
from paquete. modulo1 import sumar
print(sumar(7, 5))
```

14. Documentación sobre librerías y paquetes de clases

14.1 Tipos de librerías en programación

Hay, en términos generales, dos tipos de librerías en programación. Existen dos tipos de librerías: las **librerías caseras** o **externas**, las cuales se diferencian en si han sido creadas por uno mismo o si han sido desarrolladas por otros programadores. Sin embargo, la distinción más relevante se encuentra entre las **librerías estáticas** y las **librerías dinámicas**.

14.2 Librerías estáticas

Estas se registran en el programa como archivos ejecutables. Su función principal es esa y solo eso; una vez utilizadas, podemos eliminarlas sin preocupaciones, ya que el programa continuará operando con la funcionalidad requerida.

14.3 Librerías dinámicas

Son distintas a las estáticas pues no se copian en el programa al compilarlas. Los módulos son cargados en tiempo de ejecución, en vez de enlazarse en tiempo de compilación.

14.4 Bibliotecas en programación más usadas

Nombraremos las librerías más populares para cada uno de los lenguajes de programación. Hay librerías javascript, librerías C++, librerías python, etc.

Javascript: Jquery, Mootols, Moment.js, anime.js, ramda, D3.js, Chart.js, MathJS, Hammer.js, Glimmer.js, etc.

C++: iostream, cmath, cstring, ctime, algorithm, etc.

Python: matplotlib, seaborn, bokeh, Numpy, Scipy, Pandas, Numba, gensim, etc.

Python cuenta con una gran cantidad de librerías que se utilizan para diferentes propósitos, como ciencia de datos, aprendizaje automático, visualización de datos, desarrollo web, entre otros. A continuación, se presentan algunas de las librerías más populares en Python:

- **NumPy**: es una librería de Python para computación numérica que proporciona una estructura de datos de matriz multidimensional de alto rendimiento y herramientas para trabajar con estas matrices.
- **Pandas**: se trata de una librería en Python que se utiliza para el análisis de datos. Ofrece estructuras de datos avanzadas y herramientas que permiten la manipulación y análisis eficiente de información.
- **Matplotlib**: es una librería de visualización de datos en Python que permite crear gráficos y visualizaciones en 2D.
- **Scikit-learn**: es una librería de Python diseñada para el campo del aprendizaje automático. Ofrece una variedad de utilidades y recursos para llevar a cabo tareas como clasificación, regresión, clustering y otras actividades relacionadas con el aprendizaje automático.
- **TensorFlow**: es una librería de Python para el aprendizaje profundo que proporciona herramientas para construir y entrenar modelos de aprendizaje profundo.
- **Keras**: se trata de una biblioteca de Python dedicada al aprendizaje profundo, la cual ofrece una interfaz amigable para la construcción y entrenamiento de modelos de aprendizaje profundo.
- **Flask**: es una librería de Python para el desarrollo de aplicaciones web que permite crear aplicaciones web ligeras y escalables.
- **Django**: es una librería de Python para el desarrollo de aplicaciones web que proporciona un marco completo para construir aplicaciones web.
- **Beautiful Soup**: es una librería de Python para el análisis de datos en HTML y XML que permite analizar y extraer datos de páginas web.
- **Pygame**: es una librería de Python para el desarrollo de juegos que proporciona herramientas para crear juegos en 2D.

Estas son solo algunas de las librerías más populares en Python. Hay muchas otras librerías útiles para diferentes propósitos, y se pueden encontrar en la documentación de Python y en la comunidad de Python.

15. Resumen

En esta unidad se ha desarrollado todo lo referente a la Programación Orientada a Objetos (POO)

Se ha explicado que una clase es una plantilla que define la estructura y comportamiento de un objeto. Cómo crear objetos a partir de una clase y asignar valores a sus atributos.

Se analizan los atributos como variables asociadas a una clase u objeto, capaces de almacenar datos. También se estudian los métodos como funciones dentro de una clase, que realizan acciones y manipulan los atributos.

Se han estudiado las herencias de clases, que permiten crear nuevas clases basadas en clases existentes, heredando atributos y métodos. Se destacan las ventajas de la herencia, como la reutilización de código y la jerarquía entre las clases.

Sobrecarga de métodos y visibilidad. También se aborda la visibilidad, que controla la accesibilidad de atributos y métodos.

Otro concepto repasado ha sido el Paso de parámetro por valor y por referencia

Se ha destacado la importancia de las librerías y paquetes en Python para ampliar la funcionalidad y acelerar el desarrollo de aplicaciones. Se mencionan ejemplos de librerías populares, como NumPy, Pandas y Matplotlib.

Glosario

- **Atributos:** es una característica o propiedad asociada a un objeto, clase o estructura de datos. Es una variable que almacena información específica sobre el objeto al que pertenece.
- **Clase**: Conjunto de datos y métodos. Conjunto de reglas de creación y comportamiento de los objetos.
- **Clase hija:** Clase que es derivada directamente de otra. Decimos que la clase B es hija de la clase A si B deriva directamente de A (está conectada directamente en el árbol de jerarquías de las clases).
- **Clase padre:** La clase de la cual otra deriva directamente. Decimos que la clase A es padre de la clase B si B deriva directamente de A (está conectada directamente en el árbol de jerarquías de las clases).
- **Encapsulación:** se refiere a la ocultación de la información y la protección de los datos dentro de una clase. Consiste en agrupar los datos (atributos) y los métodos relacionados en una sola unidad, llamada objeto, y restringir el acceso directo a los datos desde el exterior de la clase.
- **Eventos:** Son aquellas acciones mediante las cuales el objeto reconoce que se está interactuando con él.
- **Herencia:** Capacidad que tienen las clases derivadas o hijas para utilizar toda la información (datos y métodos) de todas las clases superiores.
- **Herencia múltiple:** Cuando una clase hija deriva de más de una clase padre; con lo que adquiere los datos y métodos de todas las clases de las cuales deriva.
- **Indentación:** se refiere al espaciado o tabulado utilizado para estructurar y organizar el código fuente. Consiste en agregar espacios o caracteres de tabulación al principio de una línea de código para indicar su relación con bloques de código superiores o inferiores.
- **Instanciar**: Genéricamente, crear una nueva variable. En OOP, se utiliza este término para referirnos a la acción de crear un nuevo objeto.
- **Método:** Son las funciones que realizan operaciones con los datos de un objeto.
- **Mensaje:** El modo de solicitar que un objeto realice una acción es mediante un mensaje. Un mensaje normalmente activa cualquiera de los métodos de la clase a la que pertenece el objeto.
- **Objeto:** Cualquiera de las instanciaciones que hacemos de una clase. Se la puede considerar como un conjunto de datos y métodos, es decir, como un nuevo tipo de dato (como lo son los tipos carácter, lógico,

array, …). Un objeto sería una variable cuyo tipo es la clase a la que pertenece.

Tech Hub <FP> euroformac

Bibliografía

Programación orientada a objetos (2023) El Libro De Python. Available at: https://ellibrodepython.com/programacion-orientada-a-objetos.

El tutorial de python (2022) Python documentation. Available at: https://docs.python.org/es/3/tutorial/